

Experiment 11

REST API for Playing Card Collection Using Express.js

Objective

Build a RESTful API using Express.js to manage a collection of playing cards. This task helps you understand routing, handling HTTP methods, and basic data manipulation in a Node.js backend environment.

Task Description

Create an Express.js server that provides API endpoints to manage a playing card collection. The API should allow you to perform operations such as listing all cards, adding a new card (with properties like suit and value), retrieving a specific card by ID, and deleting a card by ID. Store card data in an in-memory array for simplicity. The API should follow RESTful principles and handle different HTTP methods (GET, POST, DELETE) clearly, responding with appropriate JSON data.

Purpose of This Project

The goal is not really about cards themselves. It's about learning how a backend API works using Express.js. The cards are just a simple example to practice.

What You're Learning

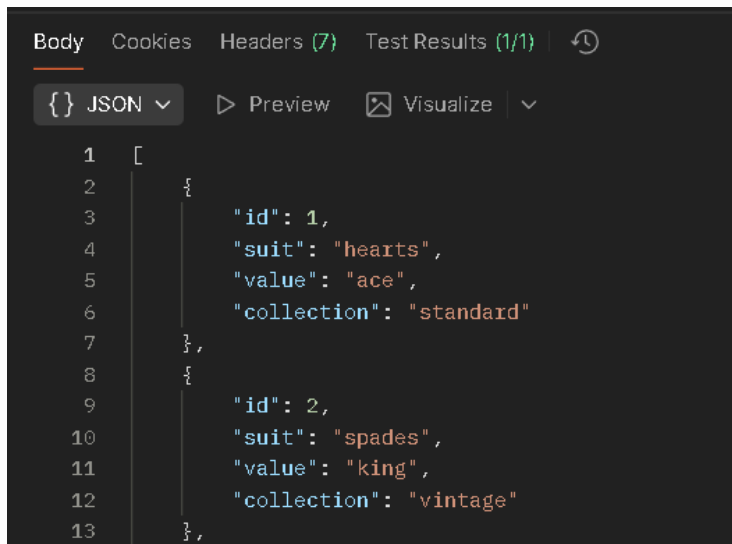
1. Express.js basics – How to set up a server and use routes (GET, POST, DELETE).
2. RESTful API principles – Understanding GET, POST, DELETE (and later PUT for updates).
3. Handling JSON data – Sending and receiving JSON between client and server.
4. In-memory database – Storing data in an array, with the option to later use real databases like MongoDB.
5. Testing APIs – Using Postman or a browser to send requests and check responses.

Real-World Connection

Instead of cards, you could manage users, products, or bank accounts. This project is a training ground to understand how larger, real-world APIs work.

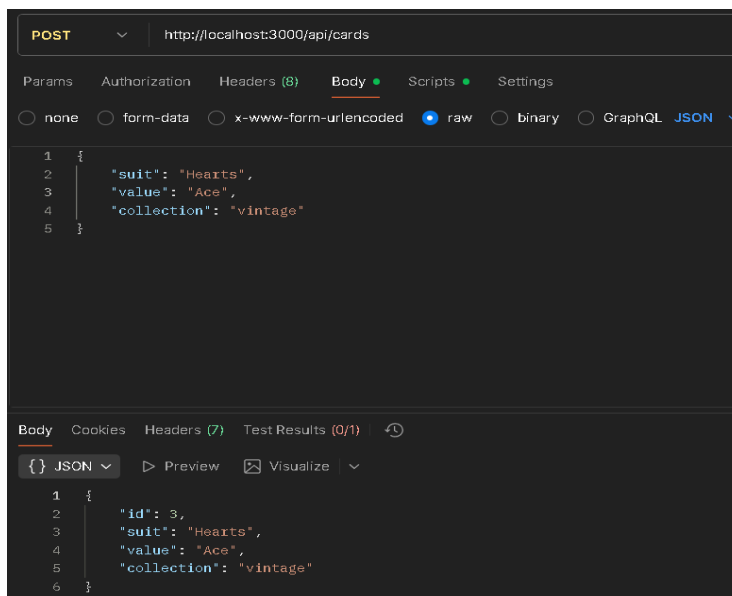
Step 4: Testing the API

1. Get all cards (initially empty)



```
1  [
2    {
3      "id": 1,
4      "suit": "hearts",
5      "value": "ace",
6      "collection": "standard"
7    },
8    {
9      "id": 2,
10     "suit": "spades",
11     "value": "king",
12     "collection": "vintage"
13   },
14 ]
```

2. Add a new card



```
POST http://localhost:3000/api/cards

{
  "suit": "Hearts",
  "value": "Ace",
  "collection": "vintage"
}
```

```
1  {
2    "id": 3,
3    "suit": "Hearts",
4    "value": "Ace",
5    "collection": "vintage"
6  }
```

3. View all cards again

```
1 {
2   "suit": "Hearts",
3   "value": "Ace",
4   "collection": "vintage"
5 }
```

Body Cookies Headers (7) Test Results (1/1) ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 [
2   {
3     "id": 1,
4     "suit": "hearts",
5     "value": "ace",
6     "collection": "standard"
7   },
8   {
9     "id": 2,
10    "suit": "spades",
11    "value": "king",
12    "collection": "vintage"
13  },
14  {
15    "id": 3,
16    "suit": "Hearts",
17    "value": "Ace",
18    "collection": "vintage"
19  }
20 ]
```

4. Get a specific card by ID

GET ▾ http://localhost:3000/api/cards

Params Authorization Headers (8) Body ● Sc

☐ none ☐ form-data ☐ x-www-form-urlencoded

```
1 {
2   "id": 1,
3   "suit": "Hearts",
4   "value": "Ace"
5 }
6
```

Body Cookies Headers (7) Test Results (1/1) ↻

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 [
2   {
3     "id": 1,
4     "suit": "hearts",
5     "value": "ace",
6     "collection": "standard"
7   },
```

5. Delete a card by ID

